

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ СИСТЕМ
УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ (ТУСУР)

Кафедра экономической математики, информатики и статистики (ЭМИС)

АНАЛИЗ СЛОЖНОСТИ АЛГОРИТМОВ

Отчёт по практической работе по дисциплине «Математическая логика и
теория алгоритмов»

Студент гр.594-2

_____ А. А. Танцеров

“ _____ ” _____ 2026 г.

Кандидат физ.-мат. наук, доцент каф.
ЭМИС

_____ И.П. Радченко

“ _____ ” _____ 2026 г.

Лабораторная работа

Анализ временной сложности алгоритмов

Цель: получение навыков оценки временной сложности алгоритмов.

1. Решите задачу.

Допустим, существует три алгоритма:

- 1) выполняет $0.01n^3$ операций на входных данных размера n ;
- 2) выполняет $n^2 + 10$ операций на входных данных размера n ;
- 3) выполняет $1000n + 10^6$ операций на входных данных размера n .

При каких значениях n первый алгоритм будет оптимальным? При $n \leq \sqrt{120}$

При каких значениях n второй алгоритм будет оптимальным? При $\sqrt{120} < n < 1618$

При каких значениях n третий алгоритм будет оптимальным? При $n \geq 1619$

2. Написать код пузырьковой сортировки и сортировки слиянием числового массива. Оценить временную сложность алгоритмов. Код представлен на листинге 1.

Сортировка пузырьком включает в себя перебор дважды, следовательно, можно утвердить, что сложность алгоритма $O(n^2)$.

Насчёт сортировки применим мастер-теорему ($T(n) = aT\left(\frac{n}{b}\right) + O(n^c)$).

Так как каждый раз мы делим на два, то $a = 2$. Происходит деление на два, следовательно, $b = 2$. Так как код линейный, имеем $c = 1$. Получим по второму ветвлению $c = \log_a b$, значит сложность алгоритма $O(n \log(n))$.

```
void bubble(vector<int> a) {  
    int n = a.size();  
    for (int i = 0; i < n - 1; i++) {  
        for (int j = 0; j < n - i - 1; j++) {  
            if (a[j] > a[j + 1]) {  
                swap(a[j], a[j + 1]);  
            }  
        }  
    }  
}
```

```

    }
}
for (int element : a) {
    cout << element << " ";
}
}

void mergeSort(vector<int> a) {
    int mid = a.size() / 2;
    vector<int> left(a.begin(), a.begin() + mid);
    vector<int> right(a.begin() + mid, a.end());
    mergeSort(left);
    mergeSort(right);
    int i = 0, j = 0, k = 0;
    while (i < left.size() && j < right.size()) {
        if (left[i] <= right[j]) {
            a[k++] = left[i++];
        }
        else {
            a[k++] = right[j++];
        }
    }
    while (i < left.size()) a[k++] = left[i++];
    while (j < right.size()) a[k++] = right[j++];
    for (int element : a) {
        cout << element << " ";
    }
}

```

Листинг 1 – Код сортировок пузырьком и слиянием